

SGLang使用

RTX 4090 推荐配置 (Qwen3-8B)

```
# RTX 4090 (24GB) 跑 Qwen3-8B 的推荐配置
python -m sglang.launch_server \
    --model-path /root/autodl-tmp/models/qwen/Qwen3-8B \
    --port 8001 \
    --host 0.0.0.0 \
    --mem-fraction-static 0.85 \      # 显存利用率 85% (4090 留一些给系统)
    --max-running-requests 16 \     # 最大并发请求数
    --context-length 4096 \         # 上下文长度 (越短越省显存)
    --chunked-prefill-size 2048 \   # 分块预填充 (降低 TTFT)
    --enable-metrics                # 开启监控指标
```

为什么“分块预填充”可以有效降低首 Token 延迟,
<https://gemini.google.com/app/370cf5a979b5bee6>

这是一个非常专业且切中要害的问题。作为技术专家,我需要先为你澄清一个行业内常见的误区:“分块预填充”(Chunked Prefill)实际上并不能降低那个“超长 Prompt”本身的首 Token 延迟(TTFT),它真正降低的是整个并发系统中“其他请求”的 TTFT。

要理解为什么,我们可以把 GPU 的计算资源想象成一家只有一张大桌子的超级厨房。

1. 传统全量预填充:可怕的“排队阻塞”(Head-of-Line Blocking)

在传统的 Continuous Batching(连续批处理)中,预填充(Prefill)是不可中断的一锤子买卖。

- 场景:假设用户 A 扔进来一个 10 万字的超级长文(如一本小说),GPU 厨房就会立刻被完全霸占,集中所有算力去算这 10 万字的 KV Cache。这个过程可能长达 5 秒钟。
- 灾难发生:就在这 5 秒内,用户 B 进来问了一句极短的话:“你好”。但在传统机制下,GPU 已经被用户 A 锁死了。用户 B 必须苦苦干等这 5 秒钟,等 A 算完,GPU 才能腾出手来处理 B 的“你好”。
- 结果:用户 B 的首 Token 延迟(TTFT)飙升到了 5 秒以上,用户体验极差。

2. 分块预填充:化整为零的“时间片轮转”

在 SGLang 等先进框架中,我们通过 --chunked-prefill-size 2048 这样的参数引入了分块预填充。

- **机制:**系统强制规定, GPU 每次最多只能处理 2048 个 Token 的预填充任务。
- **场景重现:**用户 A 的 10 万字长文进来后, 被切成了 50 块(每块 2048 个 Token)。GPU 处理完第一块(可能只需 0.1 秒)后, 会停下来检查一下队列。
- **化解危机:**此时 GPU 发现用户 B 问了句“你好”(只有 2 个 Token)。因为 GPU 现在的调度颗粒度变细了, 它会立刻把用户 B 的请求塞进下一个处理批次中。
- **结果:**用户 B 只需要等待 0.1 秒就能看到回复(TTFT 大幅降低), 而用户 A 的长文预填充则在后台分 50 次平滑地计算完毕。

3. 额外核心收益: 预填充与解码的完美混合 (Prefill-Decode Mixing)

除了解决排队阻塞, 分块预填充还解决了大模型推理的另一个死穴: 算力闲置。

- ****解码阶段(Decode)****是逐字生成的, 极度依赖显存带宽, 但对 GPU 的算力消耗很低(大量计算单元在闲置)。
- ****预填充阶段(Prefill)****是极度消耗算力的计算密集型任务。

通过分块预填充, 调度器可以在同一个 Batch 内进行****“混合双打”****: 把用户 A 切碎的 Prefill 块(耗算力)和用户 C、D 的 Decode 任务(不耗算力)拼在一起同时交给 GPU 计算。这样既填补了 Decode 时 GPU 的算力空窗期, 又保证了所有用户的生成都不会被卡顿, 实现了系统整体吞吐量和流畅度的极致平衡。